模块 2：底层模型 — 引擎的”燃料”

模块概述

> **核心问题：哪些模型能力支撑了 Agent 行为？**

底层的大语言模型（LLM）是驱动 Agent 一切行为的引擎。Agent 框架再精妙，如果底层模型无法理解代码、遵循指令、调用工具、处理长上下文，一切都是空中楼阁。

能力维度	核心问题	Agent 行为映射
代码预训练	模型为什么能写代码？	代码生成、补全、重构
指令遵循	模型如何学会听指令？	按用户意图行动
Tool Use	模型如何调用外部工具？	执行 shell、读写文件
长上下文	模型能处理多大的代码库？	理解完整项目结构
模型选择	何时用何种模型？	成本/性能权衡
基准测试	如何评估模型能力？	避免被排行榜误导

2.1 代码预训练

从 Codex 到 Claude：为什么在代码上预训练如此有效

演进路线

```
GPT-3 (2020)          通用文本生成，代□能力有限
  ▼
Codex (2021)          在 GitHub 代□上 fine-tune，HumanEval 28.8%
  ▼
ChatGPT (2022)        RLHF □□，代□能力作□"副□品"大幅提升
  ▼
GPT-4 (2023)          多模□+更大□模，HumanEval 67%→87%
  ▼
Claude 3.5 Sonnet     Constitutional AI + 大□模代□□□□，SWE-bench 49%
```

```
  ▼
Claude Opus 4/4.5     Agent ⍰代⍰能力, SWE-bench 72%+
  ▼
⍰用代⍰模型      DeepSeek Coder, Qwen2.5 Coder, StarCoder 2
```

核心趋势：**代码能力从”专用 fine-tune”走向”预训练原生”**。早期需要专门做 fine-tuning（Codex），现在顶级模型在预训练阶段就大量混入代码数据。

为什么代码是特殊的训练数据

1. 结构化与形式语义

```python
# 自然⍰言："把列表里的偶数找出来" —— ⍰⍰模糊
# 代⍰：精确无歧⍰
def filter_even(numbers: list[int]) -> list[int]:
    return [n for n in numbers if n % 2 == 0]
```

代码有严格的语法规则、类型系统和明确的执行语义，迫使模型学习精确推理而非模糊语义匹配。

2. 自带验证机制（Tests）

```python
assert filter_even([1, 2, 3, 4, 5]) == [2, 4]
assert filter_even([]) == []
```

代码可通过测试验证正确性，为 RL 提供了天然的 reward signal。

3. 多层次抽象 — 从项目架构到 token 级语法，代码在每个层次都有结构模式，让模型学习层次化推理。

Fill-in-the-Middle (FIM) 训练

标准自回归只能”从左到右”生成，但代码修改需要在上下文中间填充：

```
原始序列:  [A] [B] [C]
FIM ⍰⍰:  <PRE> [A] <SUF> [C] <MID> [B]
```

为什么 FIM 对 Agent 重要？ Claude Code 的 `Edit` 工具本质上就是 FIM——给定 `old_string`（上下文）生成 `new_string`（填充内容）。

“Code as Reasoning Substrate” 假说

在代码上预训练不仅让模型能写代码，还提升了通用推理能力：

- 代码中的 if-else、循环 → 逻辑推理能力
- 把复杂任务分解为步骤 → 规划能力
- 注释+代码模式 → Chain-of-Thought 的天然来源

```
# 代□□□数据中天然的 Chain-of-Thought:
# Step 1: 解析□入参数
args = parse_arguments(sys.argv)
# Step 2: 加□配置文件
config = load_config(args.config_path)
# Step 3: □行推理
result = model.predict(args.input)
```

这种”注释→代码”模式与”思考→行动”的 Agent 模式高度同构。

2.2 指令遵循与 RLHF

模型如何学会”听指令”

预训练后的模型是”文本续写机器”。**指令遵循训练的目标是将”续写”转变为”执行”。**

Supervised Fine-Tuning (SFT)

收集 (指令, 期望输出) 数据对，直接做 supervised learning：

数据来源	优势	劣势
人工标注	高质量、多样性	成本高、规模受限
蒸馏（Distillation）	规模大、成本低	受教师模型限制
合成数据（Self-Instruct）	可自动扩展	可能有偏差

SFT 的根本问题：**只教模型模仿”好答案”的分布，不教它区分”好”和”不好”。**

RLHF Pipeline

```
□段 1: Reward Model □□
□□□定 Prompt → 模型生成多个回复 → 人类排序 (A > B > C)
  → □□□Reward Model
  Loss = -log(σ(r(x, y_w) - r(x, y_l)))

□段 2: PPO □化
 目□: max E[R(x, y)] - β · KL(π_θ || π_ref)
  KL □□防止模型偏离□□□分布太□
```

PPO 需要同时加载 4 个模型（Policy, Reference, Reward, Value），GPU 内存需求约 SFT 的 4 倍。

Constitutional AI（Anthropic 的方法）

用一组”宪法原则”指导模型自我改进，核心流程：

1. 模型生成初始回复 → 2. 根据原则批评自身 → 3. 修改回复 → 4. 用修改后的对训练偏好模型 → 5. RLAIF（RL from AI Feedback）

CAI 的核心优势：**可扩展性**——不需要大量人类标注者，模型自身根据原则评判。

DPO：更简单的替代方案

```
RLHF:  偏好数据 → □□□RM → PPO □化 → □□模型
DPO:   偏好数据 → 直接□化 → □□模型
```

对比	RLHF (PPO)	DPO
复杂度	高（4个模型）	低（2个模型）
稳定性	不稳定，需调参	相对稳定
效果上限	通常更高	接近但略低

指令遵循为什么对 Agent 至关重要

Agent 循环中每一步都依赖精确的指令遵循：System Prompt 理解、工具定义解读、用户意图分解、错误恢复。**任何一环”不听话”，整个 Agent 循环就崩溃。**

2.3 Tool Use 能力

Function Calling：从自然语言到结构化 JSON 输出

Tool Use 的训练方式

模型在训练时学习特殊的 tool call 格式——何时触发调用、如何生成合规参数：

```
User: What's the weather in Beijing?
Assistant: <tool_call>
{"name": "get_weather", "arguments": {"city": "Beijing"}}
</tool_call>
<tool_result>
{"temperature": 22, "condition": "sunny"}
</tool_result>
Assistant: Beijing is 22°C and sunny.
```

Tool Use Protocol 完整流程

```
Step 1: System Prompt 注入工具定⍰ (JSON Schema)
Step 2: 用⍰⍰送⍰求
Step 3: 模型生成 tool_use block
Step 4: 系⍰⍰行工具，返回 tool_result
Step 5: 模型基于⍰果⍰⍰推理或再次⍰用工具
Step 6: 循⍰直到任⍰完成
```

并行工具调用

当多个调用之间无依赖时，模型可在单次回复中发起多个调用：

```
[
  {"type": "tool_use", "name": "read_file", "input": {"path": "src/auth.py"}},
  {"type": "tool_use", "name": "read_file", "input": {"path": "src/config.py"}},
  {"type": "tool_use", "name": "run_command", "input": {"command": "git log -5"}}
]
```

3 个独立操作从 3 轮变成 1 轮——显著减少延迟和成本。但模型必须正确判断哪些调用之间存在依赖。

为什么 JSON 结构化输出对自回归模型很难

挑战	说明
括号匹配	`{` 和 `}` 之间可能跨越数百 token
转义字符	`\n`，`\"` 需精确转义
Schema 合规	字段名、类型必须精确匹配定义
长字符串值	多行代码内容需在长距离内维持 JSON 格式

解决方案：

1. **Constrained Decoding**：推理时通过 JSON grammar 限制模型只能生成合法 token sequence，从根源上杜绝格式错误
2. **专项训练**：在训练数据中大量加入 JSON 格式的工具调用数据，让模型”内化”结构
3. **重试机制**：如果生成的 JSON 无法解析，将 parse error 反馈给模型要求重新生成

Claude 的 Tool Use 实现

```python
import anthropic
client = anthropic.Anthropic()

response = client.messages.create(
    model="claude-sonnet-4-20250514",
    max_tokens=1024,
    tools=[{
        "name": "read_file",
        "description": "Read the contents of a file",
        "input_schema": {
            "type": "object",
            "properties": {
                "file_path": {"type": "string", "description": "Absolute path"}
            },
            "required": ["file_path"]
        }
    }],
    messages=[{"role": "user", "content": "Read /tmp/test.py"}]
)

for block in response.content:
    if block.type == "tool_use":
        print(f"Tool: {block.name}, Input: {block.input}")
```

设计特点：工具定义是 API 参数的一等公民（非文本嵌入）、`stop_reason="tool_use"` 便于程序判断、原生并行调用支持、强类型 Schema 校验。

2.4 长上下文推理

128K/200K 窗口的技术实现与实际性能衰减

长上下文技术

RoPE Scaling：将更长序列的位置”压缩”到原训练长度的位置空间中。NTK-Aware Scaling 改进了均匀压缩的问题，高频少缩放、低频多缩放。

ALiBi：不用位置编码，直接在 attention score 上加距离惩罚：`score(i,j) = q_i·k_j - m·|i-j|`，天然支持长度外推。

YaRN：结合 NTK-aware scaling 和注意力温度补偿，只需极少量长文本 fine-tune 数据（~0.1%），在 128K 长度上保持 90%+ 质量。

“Lost in the Middle” 现象

模型对上下文中间部分的信息召回能力显著弱于开头和结尾（Liu et al., 2023）。

Agent 场景	影响	缓解策略
长 System Prompt	中间工具定义可能被忽略	关键指令放开头和结尾
多文件上下文	中间文件信息可能丢失	按相关性排序
长对话历史	早期工具结果可能被遗忘	定期总结和压缩历史

有效上下文 vs 名义上下文

```
上下文⍰度   信息⍰索率   推理⍰量（相⍰⍰）
4K            ~99%        100%
32K           ~96%        ~88%
128K          ~88%        ~70%
200K          ~82%        ~60%
```

实践建议：名义窗口的 50-60% 是有效工作区间；代码（有语法结构）比纯文本在长上下文中更不容易丢信息。

KV Cache 的内存约束

```
KV Cache 内存 = 2 × layers × heads × d_head × seq_len × dtype_size

70B 模型 + 128K 上下文 ≈ 330 GB —— 个求就需要么多内存
```

关键优化技术：

技术	压缩比	质量损失
GQA (Grouped Query Attention)	4-8x	极小
KV Cache Quantization (INT8)	2-4x	小
PagedAttention (vLLM)	减少碎片浪费	无

2.5 模型选择策略

Opus vs Sonnet vs Haiku：能力/成本/延迟三角

详细对比表

维度	Opus 4	Sonnet 4	Haiku 3.5
输入价格	$15/M tokens	$3/M tokens	$0.80/M tokens
输出价格	$75/M tokens	$15/M tokens	$4/M tokens
相对延迟	慢 (1x)	中 (2-3x faster)	快 (5-8x faster)
SWE-bench Verified	72%+	65%+	~40%
复杂推理	最强	强	一般
工具使用可靠性	极高	高	中等

成本直观对比

```
⍰景: Agent 修复中等⍰度 bug (~50K input, ~10K output)

Opus:   $0.75 + $0.75 = $1.50
Sonnet: $0.15 + $0.15 = $0.30
Haiku:  $0.04 + $0.04 = $0.08

Opus 是 Haiku 的 ~19 倍成本
```

多模型切换策略

核心思路：**简单任务用快速模型，复杂任务用强力模型。**

```python
# 方式 1: 静⍰路由
ROUTING = {
    "completion": "haiku",      # 自⍰⍰全 → 快速
    "explain": "sonnet",        # 解⍰代⍰ → 中等
    "debug": "opus",            # 复⍰⍰⍰ → 强力
}

# 方式 2: ⍰⍰策略 (Cascading)
def cascading_solve(task):
    result = try_with_model("haiku", task)
    if result.confidence > 0.9:
        return result
    result = try_with_model("sonnet", task)
    if result.confidence > 0.8:
        return result
    return try_with_model("opus", task)
```

何时用哪种模型

场景	推荐	理由
IDE 内联补全	Haiku	延迟要求极高（< 200ms）
单文件 Bug 修复	Sonnet	不需要 Opus 级推理
跨文件重构	Opus	需要理解项目级依赖
从 Issue 到 PR	Opus	完整的理解-规划-执行链
文档生成	Haiku/Sonnet	格式化输出，不需深度推理

成本优化策略

Prompt Caching：System Prompt + 工具定义在多轮对话中不变，缓存命中可减少 90% 重复成本。

分层模型的节约效果：

```
1000 个任?/天:
- 60% ?? (Haiku):  600 × $0.08 = $48
- 30% 中等 (Sonnet):  300 × $0.30 = $90
- 10% 复? (Opus):   100 × $1.50 = $150
??: $288/天 vs  全用 Opus: $1,500/天  → ?省 81%
```

2.6 基准测试的真相

SWE-bench, HumanEval：测了什么，没测什么，怎么被刷分

HumanEval：简单函数补全

```
# 典型?目：?定 docstring，?全函数体
def has_close_elements(numbers: List[float], threshold: float) -> bool:
    """Check if any two numbers are closer than threshold."""
    # 模型生成 →
```

局限：只有 164 题、难度低、已被饱和（顶级模型 95%+）、数据泄露风险高。

MBPP → APPS → CodeContests：逐步升级

难度阶梯从基础算法到竞赛编程，但仍是”算法能力”测试，非”软件工程能力”。

SWE-bench：真实世界的 GitHub Issue

从真实开源仓库提取 Issue，要求 Agent 生成修复补丁并通过测试。**当前评估代码 Agent 最重要的基准。**

版本	题目数	特点
SWE-bench Full	2,294	完整数据集，部分有歧义
SWE-bench Lite	300	精选子集
SWE-bench Verified	500	人工验证，最高质量，当前主流

SWE-bench 为什么比 HumanEval 重要得多：

1. **真实任务**：来自真实的开源项目维护，不是人造题目
2. **项目级别**：需要理解整个代码库的结构和依赖关系，不是单个函数
3. **需要工程能力**：定位 Bug 位置 → 理解代码逻辑 → 生成正确修复
4. **有明确验证**：通过测试套件验证，客观且可重复

基准测试如何被”刷分”

1. **数据污染**：题目出现在训练数据中，模型”记住”而非”推理”
2. **格式过拟合**：针对特定输出格式（如 diff patch）专门优化
3. **Test-time Compute 堆量**：100 次采样取最优，pass@1 vs pass@100 差异巨大
4. **专用 Prompt Engineering**：不可迁移的 prompt 优化

基准测试没测到什么

```
✅ 覆盖：⍰定描述写函数、修复已知 bug、通⍰已有⍰⍰
❌ 未覆盖：
  - 代⍰⍰⍰与向后兼容          - 架⍰⍰⍰与模式⍰⍰
  - 重⍰（改⍰⍰不改行⍰）       - ⍰⍰⍰⍰（⍰写⍰⍰而非通⍰⍰⍰）
  - 性能⍰化                  - 从模糊需求推⍰精确需求
  - 从⍰⍰⍰象定位根因          - 写出可⍰⍰的代⍰
```

启示：不要仅凭基准分数选模型。应该在自己的实际场景中做评估：

```
# ⍰用的模型⍰估框架
evaluation_dimensions = {
    "correctness":          "生成的代⍰是否正确",
    "instruction_following": "是否精确遵循了指令",
    "tool_use_reliability":  "工具⍰用格式是否始⍰正确",
    "context_utilization":   "是否有效利用了上下文信息",
    "error_recovery":        "遇到⍰⍰⍰能否自主恢复",
    "code_quality":          "生成代⍰的可⍰性和⍰⍰性",
    "efficiency":            "完成任⍰所需的⍰次和 token 数",
}
```

关键论文导读

Codex (Chen et al., 2021)

Evaluating Large Language Models Trained on Code

首次大规模评估 LLM 代码生成能力；提出 HumanEval 和 pass@k 指标；展示了 fine-tune on code 的有效性。**Agent 启示**：多次采样 + 验证 = 提高成功率——这正是 Agent “试错” 循环的理论基础。

InstructGPT / RLHF (Ouyang et al., 2022)

Training Language Models to Follow Instructions with Human Feedback

完整 RLHF pipeline（SFT → RM → PPO）；证明 1.3B 对齐模型可优于 175B 未对齐模型。**Agent 启示**：指令遵循是 Agent 可靠性基础；小而对齐 > 大而未对齐。

Constitutional AI (Bai et al., 2022)

Constitutional AI: Harmlessness from AI Feedback

用 AI 反馈替代人类反馈（RLAIF），原则驱动的自我改进。**Agent 启示**：“自我批评 + 修正” 模式可用于 Agent 的自我改进。

Toolformer (Schick et al., 2023)

Toolformer: Language Models Can Teach Themselves to Use Tools

首次展示 LLM 可自学工具使用——模型自动判断何时调用工具、如何调用，无需人工标注的工具调用数据。核心方法：在文本中插入候选 API 调用 → 执行并获取结果 → 比较有/无 API 调用时的 perplexity → 保留有效的调用作为训练数据。**Agent 启示**：模型学会”何时用工具 vs 直接回答”——这是 Agent 决策的基础能力。

实操环节

同一编程任务，三种模型，对比行为差异与成本

```
"""
□操：模型□比□□
□同一个 Bug □□任□分别□用 Opus/Sonnet/Haiku，□察差异
"""
import anthropic, time

client = anthropic.Anthropic()

TASK = """
Review the following Python code and identify all bugs.
For each bug, explain the issue and provide the fix.

```python
def merge_sorted_lists(list1, list2):
    result = []
    i = j = 0
    while i < len(list1) and j < len(list2):
        if list1[i] <= list2[j]:
            result.append(list1[i])
            i += 1
        else:
            result.append(list2[j])
            # Bug 1: 缺少 j += 1 → 无限循□
    result.extend(list1[i:])
    result.extend(list2[j:])
    return result

def find_median(sorted_list):
    n = len(sorted_list)
    mid = n // 2
    if n % 2 == 0:
        return (sorted_list[mid] + sorted_list[mid - 1]) / 2
    else:
        return sorted_list[mid + 1]  # Bug 2: □□□sorted_list[mid]

def binary_search(arr, target):
    left, right = 0, len(arr)
    while left < right:
        mid = (left + right) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid   # Bug 3: □□□left = mid + 1
        else:
            right = mid
    return -1

“  “”
```
```

MODELS = [(“claude-opus-4-20250514”, {“input”: 15.0, “output”: 75.0}), (“claude-sonnet-4-20250514”, {“input”: 3.0, “output”: 15.0}), (“claude-haiku-4-5-20251001”, {“input”: 0.80, “output”: 4.0}),]

for model_id, pricing in MODELS: start = time.time() resp = client.messages.create(model=model_id, max_tokens=2048, messages=[{“role”: “user”, “content”: TASK}]) elapsed = time.time() - start cost = (resp.usage.input_tokens * pricing[“input”] + resp.usage.output_tokens * pricing[“output”]) / 1_000_000

```
print(f"\n{'='*50}")
print(f"{model_id}")
print(f"Time: {elapsed:.1f}s | Cost: ${cost:.4f}")
print(f"Tokens: {resp.usage.input_tokens} in / {resp.usage.output_tokens} out")
print(f"\n{resp.content[0].text[:600]}...")
```

```

### 观察维度

| 维度 | 关注点 |
|---|---|
| Bug 发现数量 | 三个模型分别发现几个？有遗漏吗？ |
| 分析深度 | 只指出 bug 还是解释了为什么是 bug？ |
| 修复质量 | 是否考虑了边界情况？ |
| 成本效率 | 性价比最高的是哪个？ |

预期：Opus 找到全部 3 个 bug 并给出深度分析；Sonnet 找到全部并解释清晰；Haiku 可能遗漏 1 个较隐蔽的 bug，但速度最快成本最低。

## 进阶实验：加入 Tool Use

在上述对比基础上，尝试给三个模型提供 `read_file`、`write_file`、`run_tests` 三个工具，观察 Agent 循环行为差异：

| 行为维度 | Opus | Sonnet | Haiku |
|---|---|---|---|
| 第一步行动 | 读文件了解全貌 | 读文件 | 可能直接给修复 |
```

行为维度	Opus	Sonnet	Haiku
工具调用次数	3-4次（读→改→测→确认）	2-3次	1-2次
错误恢复	测试失败后分析并重试	测试失败后重试	可能不运行测试
调用格式正确率	始终正确	始终正确	偶尔格式错误

这种差异直接映射到生产环境中 Agent 的可靠性——工具调用格式错误率哪怕只有 5%，在 10 轮的 Agent 循环中就有 ~40% 的概率至少出一次错。

本模块小结

核心要点

1. **模型能力决定 Agent 上限** — 再好的框架也无法突破底层模型的天花板
2. **对齐比规模更重要** — 对齐良好的中等模型 > 未对齐的大模型
3. **Tool Use 是 Agent 的分水岭** — 没有可靠的 Tool Use，Agent 只是聊天机器人
4. **长上下文有代价** — 不要盲目塞满窗口，注意 “lost in the middle”
5. **基准测试是参考不是真相** — 在自己的场景中做评估才是关键

思考题

基础理解

1. 为什么在代码上预训练可以提升模型的通用推理能力？从训练数据特点分析。
2. 解释 RLHF 中 KL 散度惩罚项的作用。去掉会怎样？
3. 自回归模型生成 JSON 结构化输出时面临哪些挑战？

深度思考

1. Constitutional AI 与传统 RLHF 各有什么优劣？若训练代码 Agent 专用模型，你选哪种？
2. “Lost in the Middle” 对 Agent 的 System Prompt 设计有什么影响？设计一个优化策略。

3. 如果让你设计一个新的基准测试来评估编程 Agent，你会包含哪些维度？

实践应用

1. 为 10 人团队部署 Agent 服务（每人每天 50 次使用），设计多模型路由策略使月成本 < $500。
2. 比较 Opus 运行 1 次 vs Haiku 运行 10 次取最佳结果。何时前者更优？何时后者更优？

> **下一模块预告**：模块 3 将进入 Agent 架构的核心——**提示工程与上下文管理**。如何通过精心设计的 System Prompt 和上下文策略，将底层模型的能力最大化释放。